4. If the two agree, accept the message. If they do not, discard what has been assembled and resume the search from the byte following the false header, since what was taken for a header was evidently something else.

Once a message has been accepted, the Python program is to reconstruct the reading from its two data bytes as `(upper « 8) | lower`, convert that reading to the corresponding potentiometer

interval; that function reads whatever bytes are already waiting, passes them to the decoder written in the previous stage, and returns immediately. This is the same arrangement used in the previous laboratory, where the state of a switch was examined on each pass through the loop rather than waited upon; only the peripheral has changed.

Bytes that arrive between one call and the next are not lost. They accumulate in a buffer maintained by the operating system, and the number waiting may be obtained from `in_waiting`.[2] All of them should be read on each call and passed to the decoder, which will assemble whatever complete messages they contain and retain the remainder. Reading a fixed number of bytes instead, on the assumption that it constitutes one message, causes the Python program to fall progressively further behind. The arrangement is shown in Figure 6.

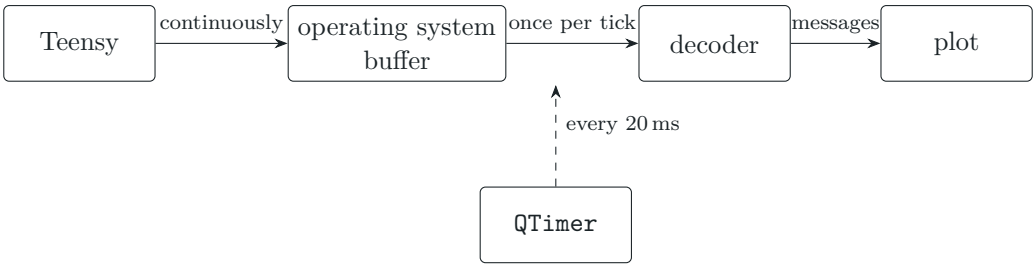


**Figure 6:** Data pipeline. Bytes arrive from the sketch continuously and accumulate in the buffer, from which every byte waiting is removed at each tick of the timer and passed to the decoder. The rate of arrival and the rate of reading are therefore independent, and the buffer holds the difference.

To confirm the necessity of this arrangement, replace the timer with a loop that waits for the next message and returns only when one has arrived. Attempt to move the window while the Python program runs, and record the result. Restore the timer afterwards.

### Questions to be answered

**Q7.1** What happens to the window while the Python program waits for the next message, and why?

## 8 Experiment 06: Sending commands from the computer to the board

The direction of travel is now reversed. In this stage the board only receives: the LED is set solely by the dial in the graphical interface, and the sketch sends nothing at all. The sampling of `A0` is removed from the sketch altogether, so that the plot built in Section 7 remains empty throughout this stage; both directions are used together in Section 9.

### 8.1 Objective

To set the brightness of the LED from a dial in the graphical interface.

### 8.2 Circuit

No change is made to the circuit. The potentiometer remains wired to `A0`, but is not used in this stage and is read again in Section 9.

[2] `https://pyserial.readthedocs.io`

## 8.3 Observations

### 8.3.1 Transmitting commands

The JEDI format of Section 1.5 serves in either direction, and the same two header bytes, size byte and checksum are used for a command as for a reading. Only the data bytes differ.

The brightness is commanded here as a twelve-bit number, in the range 0 to 4095. Such a value does not fit in one byte, and is therefore carried in two, exactly as a reading was in Section 6: the upper byte, obtained as `value » 8`, followed by the lower byte, obtained as `value & 0xFF`. A command thus has two data bytes, and the upper of them never exceeds 15.

Twelve bits are also more than `analogWrite` accepts by default. The sketch must therefore call `analogWriteResolution(12)`, as Section 5 anticipated, after which `analogWrite` accepts values from 0 to 4095 and no scaling of the commanded value is required.

Add a `QDial` to the interface, with its range set to 0 to 4095. Whenever its value changes, the Python program is to construct such a command and write it to the serial port.

### 8.3.2 Receiving commands on the board

The sketch must read these messages without waiting for them. Commands arrive only when the dial is moved, and a sketch that waited for one would be unable to do anything else in the meantime – the same difficulty met on the computer in Section 7, and avoided there by a timer. `Serial.available()` reports the number of bytes already received; if it is zero the loop proceeds, and if it is not, those bytes are passed to a decoder identical in structure to the one written in Python.

Once a complete message has been decoded and its checksum found to agree, the sketch is to reconstruct the twelve-bit value as `(upper « 8) | lower` and set the brightness to it. A message whose checksum does not agree is discarded, as on the computer.

Move the dial slowly through its range and confirm that the LED responds.

**Questions to be answered**

**Q8.1** Why is the commanded value carried in two bytes rather than one, and what is the largest value the upper byte can take?

**Q8.2** What would the LED do were `analogWriteResolution(12)` omitted from the sketch, the dial still being moved through its whole range?

# 9 Experiment 07: Bidirectional communication

## 9.1 Objective

To transmit readings and receive commands simultaneously, to place the control logic on the computer rather than on the board, and to measure the latency this introduces.

### 9.2 Circuit

Add a push-button between digital pin `3` and `GND`, configured with the internal pull-up as in the previous laboratory. Add a second LED of a different colour, in series with a current-limiting resistor, between digital pin `4` and `GND`.

### 9.3 Observations

#### 9.3.1 LED control

The original LED lightining logic remains the same as the one before. The potentiometer controls the original LED's brightness, and this potentiometer value is transmitted to the PC and dispalyed on the Python program's plotter.

The green LED's ON/OFF state is controlled by the push button. The LED is off when the button is not pressed. When the button is not pressed the dial on the graphical interface is disabled. When the button is pressed, the dial is enabled and its position determines the brightness of the green LED. The potentiometer does not control the green LED's brightness.

**Table 7:** Data bytes of the message sent from the board, and the stage at which each was introduced.

| Byte | Contents | Introduced in |
|---|---|---|
| 1–2 | the reading, upper byte first | Section 6 |
| 3 | the state of the button | |

#### 9.3.2 Simultaneous operation

Confirm that readings continue to be plotted while commands are being transmitted, and that neither direction impedes the other. Under board control, move the dial and record the state of the LEDs and the value displayed by the interface; repeat under computer control, turning the potentiometer instead. Then press the button while the dial is in continuous motion.

## 10 Deliverables

Submit the following:

1. The completed tables: Tables 2 and 3 from Section 3, Table 4 from Section 4, Table 5 from Section 5, and Table 6 from Section 6.
2. The plot of the reported value against the measured wiper voltage, from Section 4.
3. The screenshot of the serial monitor called for in Q6.1.
4. The sketch and the Python program in their final form, with comments. Where a stage altered either of them in a way worth showing, include that version also, labelled with the section it belongs to.
5. Written answers to every question posed in the Observations of each stage, each identified by its number. The number gives the section in which the question appears: **Q4.1**, for instance, is the first question of Section 4.

## References

[1] PJRC, *Teensy 4.1 Development Board*. https://www.pjrc.com/store/teensy41.html

[2] *pySerial Documentation*. https://pyserial.readthedocs.io

[3] *QTimer Class*. https://doc.qt.io/qt-6/qtimer.html

[4] *PyQtGraph Documentation*. https://pyqtgraph.readthedocs.io